# From Observability to Verifiability: An Execution Evidence Architecture for Agentic Software Systems

Anonymous Author(s)

## Abstract

Agentic software systems are increasingly expected to invoke tools, consume budgets, and produce externally consequential outputs, yet most current runtimes still expose execution primarily through logs and traces. Those artifacts are useful for observability, but they do not by themselves provide a bounded, reviewable, and independently checkable account of what was intended, what was authorized, what was executed, and what evidence was exported. This paper frames that mismatch as an execution evidence problem rather than a logging problem.

We present an execution evidence architecture for agentic software systems and instantiate it as a reproducible reference path in a submitted artifact package omitted here for double-blind review. The architecture links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpointing, execution-integrity validation, and bounded audit receipts. The controlled evaluation directly tests the latter four evidence-preserving stages through a deterministic 16-task suite, a unified five-file export contract, a third-party bundle review script, minimal live CrewAI and LangChain baselines, paired same-framework comparisons in both frameworks with and without the evidence layer, four architectural ablations, and a small falsification suite for negative controls.

Across the reported evaluation artifacts, the full evidence-chain configuration improves average explicitness from 1.0 to 5.0, replayability from 3.0 to 5.0, tamper sensitivity from 0.0 to 4.81, audit boundedness from 2.0 to 5.0, and evidence-stage exposure from 1.0 to 5.0 relative to the local trace-oriented baseline. The minimal live CrewAI and LangChain baselines each reach replayability 4.0, but leave the other evidence dimensions at observability-like levels. In both same-framework paired comparisons, wrapping the identical live runtime with the evidence layer raises the corresponding evidence metrics to 5.0, 5.0, 4.81, 5.0, and 5.0. Ablation results show targeted degradation when intent capture, governance persistence, integrity validation, or receipt export is removed. Under an eight-scenario falsification suite, the updated review contract rejects cross-run replay mismatches, cross-bundle receipt swaps, receipt-digest forgeries, payload tampering, false tamper claims, and policy-bypass claims, yielding a 1.0 detection rate across all derived bundles and showing that the contract is not limited to field-presence checks. In this controlled harness, the results support a bounded claim that observability surfaces and verifiability surfaces are not interchangeable.

*ASE 2026, Munich, Germany*
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

## CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; **Software creation and management**; *Formal software verification*; • **Computing methodologies** → *Artificial intelligence*.

## Keywords

agentic software systems, execution evidence, audit receipts, verifiability, runtime governance

**ACM Reference Format:**
Anonymous Author(s). 2026. From Observability to Verifiability: An Execution Evidence Architecture for Agentic Software Systems. In *The 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026)*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Agentic software systems increasingly execute software tasks, invoke tools, consume budgets, and produce outputs that matter beyond the runtime boundary. As these systems move from demos to operational workflows, the engineering question is no longer only whether an agent can complete a task. It is also whether the resulting run can be independently reviewed, replayed, and trusted by a third party.

Current practice usually answers this need with observability artifacts such as logs, callback streams, trace records, and framework-specific execution histories. These artifacts are valuable for debugging and operational visibility, but they are not the same as execution evidence. A trace can show that something happened without making the requested action explicit, without preserving a governance decision, without exposing whether execution integrity was checked, and without exporting a bounded receipt that a reviewer can inspect without reconstructing the full runtime context.

We call this mismatch the *execution evidence gap*. An operator may be able to inspect a log and infer what likely happened, but an independent reviewer still lacks a stable artifact contract for answering four basic questions: was intent captured, was policy checked, was execution verified, and was a receipt exported? When those questions are left implicit, a run can be observable without being verifiable.

This paper presents an execution evidence architecture for agentic software systems. The proposal is not a new orchestration framework. Instead, it is a composable systems layer that links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpoints, execution-integrity validation, and bounded audit receipts. The architecture makes runs exportable as reviewable evidence bundles while directly evaluating only the latter four evidence-preserving stages.

We instantiate the design in a submitted artifact package omitted here for double-blind review. The artifact package includes a

deterministic 16-task suite, a unified five-file export contract, a third-party review script, a trace-oriented baseline, minimal live CrewAI and LangChain baselines, same-framework paired comparisons, four ablations, and an eight-scenario falsification suite. Within the current exported-bundle contract, the evidence-chain configuration improves every evaluated evidence-shape metric over the trace-oriented baseline, while the live framework baselines primarily improve replayability without recovering the missing evidence semantics.

The contributions are threefold:

- It formulates execution evidence as a software systems problem that is distinct from runtime observability.
- It defines a five-stage execution evidence architecture while directly evaluating the latter four evidence-preserving stages as an operational review contract.
- It implements an artifact-grounded evaluation with live-framework baselines, paired comparisons, ablations, and falsification checks showing that bounded reviewability is not recovered by observability surfaces alone.

## 2 Method

The goal of this work is to define and evaluate a minimal execution evidence architecture for agentic software systems. The target problem is narrower than model correctness and broader than debugging: how should runtime artifacts be organized so that an independent reviewer can determine whether intent was captured, policy was checked, execution was verified, and evidence was exported? The method therefore centers on exported artifact contracts rather than hidden runtime state.

### 2.1 Execution evidence architecture

We define the architecture as a five-stage evidence chain:

(1) persona projection,
(2) intent/action/result objects,
(3) governance checkpoint,
(4) execution-integrity validation,
(5) bounded audit receipt.

Persona projection provides the actor boundary but is treated here as an architectural precondition rather than a separate empirical target. Intent/action/result objects expose the requested task, operational step, and terminal state. Governance persistence makes blocked runs evidence-bearing instead of silently absent. Execution-integrity validation records replay- or tamper-relevant status. The bounded receipt compresses the run into a stable review surface. Together these stages convert framework-specific observability into a portable evidence bundle. Figure 1 summarizes the path.

### 2.2 Reference path

The architecture is instantiated as a composable reference path in a submitted artifact package omitted here for double-blind review. The implementation keeps the evaluation logic centered in a deterministic local harness so that every condition is reproducible from the included task specifications and scripts.

For every task run, the harness exports exactly five JSON files:

- `intent.json`,

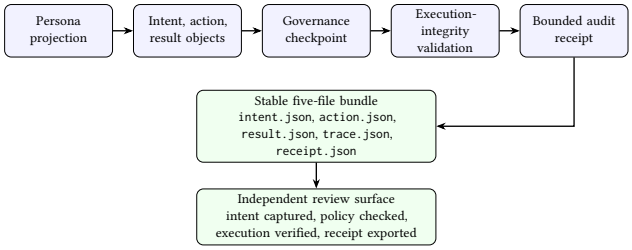


**Figure 1: Five-stage architectural view. The controlled evaluation in this paper directly validates the latter four evidence-preserving stages while treating persona projection as an architectural precondition.**

- `action.json`,
- `result.json`,
- `trace.json`,
- `receipt.json`.

These files are written under the stable layout `artifacts/runs/<task_id>/<mo` The contract is shared across all conditions, including baselines and ablations, which makes evidence shape directly comparable.

### 2.3 Task suite

The evaluation harness includes 16 machine-readable tasks grouped into five categories:

- four read-only query tasks,
- four tool-call tasks,
- three budget-constrained tasks,
- three approval-required tasks,
- two tamper scenarios.

The mix of task classes is important because an evidence architecture should represent both positive and negative execution paths. In particular, the budget, approval, and tamper tasks test whether blocked and invalid runs appear as explicit outcomes rather than as absent output.

### 2.4 Experimental conditions

We evaluate ten experimental conditions: a local trace-oriented baseline; two minimal live framework baselines (`external_baseline` for CrewAI and `langchain_baseline` for LangChain); two same-framework wrapped paths that preserve the live runtime while adding the evidence layer; the full evidence-chain condition; and four single-stage ablations (`no_intent`, `no_governance`, `no_integrity`, and `no_receipt`). The live framework baselines preserve framework-native execution evidence but not explicit governance, integrity, or receipt semantics. The paired wrapped modes isolate the effect of the evidence layer while holding the underlying runtime fixed.

### 2.5 Review logic and metrics

A third-party review script consumes a run directory and answers four yes/no questions:

(1) was intent captured,
(2) was policy checked,
(3) was execution verified,
(4) was a receipt exported.

The script emits machine-readable verdicts, an overall status, missing-field explanations, and a short summary. It also recomputes bundle identity consistency and stored digests rather than trusting exported status labels alone. This review step operationalizes the minimum independent inspection workflow that the architecture is intended to support.

We compare conditions using five deterministic, rule-based metrics derived from actual generated artifacts:

- **explicitness**: whether intent, policy, and outcome are directly inspectable;
- **replayability**: whether the run preserves enough structure for replay-oriented checking;
- **tamper sensitivity**: whether integrity-relevant violations appear as explicit outcomes;
- **audit boundedness**: whether the review surface is compact and purpose-oriented;
- **evidence-stage exposure**: how completely the harness exposes the intended evidence stages (field name: `integration_surface`).

Some parts of these metrics are directly measurable from the exported fields, while others are scored by transparent 0–5 rules documented in the submitted artifact package. The method does not report semantic-quality scores or invented empirical values.

## 3 Evaluation Setup

All measurements in this draft come from the submitted artifact package. The evaluation asks whether execution evidence should be treated as a distinct systems layer rather than as an extension of runtime logging. The task suite is intentionally heterogeneous: read-only and tool-call tasks cover ordinary successful runs, budget and approval tasks exercise policy gating, and tamper scenarios exercise integrity-sensitive outcomes. Because the same 16 task specifications are used in every condition, score differences are attributable to the exported evidence surface rather than to different workloads.

The review script distinguishes among several outcome classes. A run may be marked *pass* when all four evidence questions are satisfied, *policy_blocked* when a checkpoint blocks the task but still leaves reviewable evidence, *tamper_detected* when the integrity layer detects mismatch, or *partial* when one or more evidence-chain capabilities are missing. In the full evidence chain, the artifact summaries show 11 review-passing completed runs, 3 policy-blocked runs, and 2 tamper-detected runs across the 16-task suite.

### 3.1 Falsification checks

To reduce the risk that the review contract merely accepts the study's own positive cases, the artifact package also derives an eight-scenario falsification suite from previously reviewable bundles. Seven scenarios begin from the 11 evidence-chain bundles that the review script already marks as *pass*; the eighth (`policy_bypass_claim`) begins from the 3 evidence-chain bundles that originally reviewed as *policy_blocked*.

Single-surface removals such as missing intent and missing policy are rebound so that the mutated bundle stays internally consistent except for the targeted capability. Receipt forgery, payload tampering, cross-run replay mismatch, and cross-bundle receipt swap exercise strengthened bundle-identity and digest-binding checks rather than trusting stored status fields. The policy-bypass claim scenario exercises cross-file coherence directly by rewriting a blocked bundle to claim successful completion while leaving the persisted blocked policy decision in place. These checks are reported separately from the main baseline and ablation comparisons because they do not measure runtime quality; they measure whether the independent review contract can reject internally contradictory or transplanted artifacts.

**Table 1: Same-framework paired comparisons under the live CrewAI and LangChain paths. In both frameworks the baseline preserves only the default framework-shaped observability surface, while the paired condition wraps that same runtime with the full evidence chain.**

| Framework | Mode | Intent | Policy | Exec. verified | Receipt | Explicitness | Replayability | Tamper | Audit | Stage exposure |
|---|---|---|---|---|---|---|---|---|---|---|
| CrewAI | CrewAI baseline | 0/16 | 0/16 | 0/16 | 0/16 | 1.0 | 4.0 | 0.0 | 2.0 | 1.0 |
| CrewAI | CrewAI + evidence chain | 16/16 | 16/16 | 11/16 | 16/16 | 5.0 | 5.0 | 4.81 | 5.0 | 5.0 |
| LangChain | LangChain baseline | 0/16 | 0/16 | 0/16 | 0/16 | 1.0 | 4.0 | 0.0 | 2.0 | 1.0 |
| LangChain | LangChain + evidence chain | 16/16 | 16/16 | 11/16 | 16/16 | 5.0 | 5.0 | 4.81 | 5.0 | 5.0 |

## 4 Results

### 4.1 Baselines and paired comparisons

Table 2 provides the quantitative anchor for the local trace baseline, the external baseline, the four ablations, and the full evidence chain. Within the current exported-bundle contract, the evidence-chain configuration improves every evaluated evidence-shape dimension over the trace-oriented baseline. Intent capture, policy checking, and receipt export move from zero positive review verdicts in the baseline to 16 of 16 tasks in the evidence chain. The reported *execution verified* count is 11 of 16 because the full suite also contains 3 reviewable policy-blocked outcomes and 2 explicit tamper-detected outcomes.

The local trace baseline reaches replayability 3.0 but leaves explicitness, tamper sensitivity, audit boundedness, and evidence-stage exposure at 1.0, 0.0, 2.0, and 1.0. The minimal live CrewAI baseline raises replayability to 4.0 by preserving a richer execution surface, but it still leaves the missing evidence semantics unresolved. These baseline results support the paper's narrower claim: observability surfaces can improve inspectability without recovering the bounded review contract.

Table 1 tightens the argument by keeping the live runtime fixed inside two framework ecosystems. In both CrewAI and LangChain, the baseline preserves only the default framework-shaped observability surface, while the paired condition wraps that same live path with explicit intent capture, governance persistence, integrity validation, and bounded receipt export.

The paired result is consistent across both frameworks. The default CrewAI and LangChain baselines each remain at 1.0 explicitness, 4.0 replayability, 0.0 tamper sensitivity, 2.0 audit boundedness, and 1.0 evidence-stage exposure. The wrapped conditions in both frameworks rise to 5.0, 5.0, 4.81, 5.0, and 5.0 while preserving the same live runtime path within their respective ecosystems. This does not eliminate all external-validity concerns, because both integrations remain minimal and local, but it materially reduces the claim that the paper only wins by comparing a custom local harness against a single weaker proxy.

**Table 2: Seven-mode comparison across the baseline, external baseline, four ablations, and the full evidence chain. Scores are average 0–5 values; counts report positive review verdicts across 16 tasks.**

| Mode | Explicitness | Replayability | Tamper | Audit | Stage exposure | Intent | Policy | Exec. verified | Receipt |
|---|---|---|---|---|---|---|---|---|---|
| Baseline | 1.0 | 3.0 | 0.0 | 2.0 | 1.0 | 0/16 | 0/16 | 0/16 | 0/16 |
| External baseline | 1.0 | 4.0 | 0.0 | 2.0 | 1.0 | 0/16 | 0/16 | 0/16 | 0/16 |
| No intent | 3.0 | 4.0 | 4.81 | 4.0 | 4.0 | 0/16 | 16/16 | 11/16 | 0/16 |
| No governance | 2.0 | 5.0 | 5.0 | 4.0 | 4.0 | 16/16 | 0/16 | 14/16 | 0/16 |
| No integrity | 5.0 | 5.0 | 0.0 | 5.0 | 4.0 | 16/16 | 16/16 | 0/16 | 16/16 |
| No receipt | 4.0 | 4.0 | 4.81 | 2.0 | 4.0 | 16/16 | 16/16 | 11/16 | 0/16 |
| Evidence chain | 5.0 | 5.0 | 4.81 | 5.0 | 5.0 | 16/16 | 16/16 | 11/16 | 16/16 |

**Table 3: Negative-control bundle checks derived from review-passing and policy-blocked evidence-chain runs. Each scenario is re-reviewed with the same independent bundle contract used in the main study.**

| Scenario | Source | Targeted failure | Detected | Intent | Policy | Exec. verified | Receipt |
|---|---|---|---|---|---|---|---|
| Missing intent | Pass | Intent | 11/11 | 0/11 | 11/11 | 11/11 | 11/11 |
| Missing policy | Pass | Policy | 11/11 | 11/11 | 0/11 | 11/11 | 11/11 |
| Forged receipt | Pass | Receipt | 11/11 | 11/11 | 11/11 | 11/11 | 0/11 |
| Payload tamper | Pass | Execution + receipt | 11/11 | 11/11 | 11/11 | 0/11 | 0/11 |
| False tamper claim | Pass | Tamper claim | 11/11 | 11/11 | 11/11 | 0/11 | 11/11 |
| Cross-run replay mismatch | Pass | Replay chain | 11/11 | 11/11 | 11/11 | 0/11 | 11/11 |
| Cross-bundle receipt swap | Pass | Receipt binding | 11/11 | 11/11 | 11/11 | 11/11 | 0/11 |
| Policy bypass claim | Policy-blocked | Policy/result coherence | 3/3 | 3/3 | 3/3 | 3/3 | 3/3 |

## 4.2 Ablation results

Table 2 summarizes the seven-mode comparison. The ablations show targeted degradation rather than uniform collapse.

Removing intent capture lowers explicitness from 5.0 to 3.0 and evidence-stage exposure from 5.0 to 4.0. Removing governance lowers explicitness to 2.0, eliminates policy-checked reviews, and changes the suite behavior by allowing formerly blocked tasks to complete. Removing integrity preserves explicitness and audit boundedness but drives tamper sensitivity and execution verification to 0.0 and 0 of 16. Removing receipt export preserves most other capabilities but drops audit boundedness from 5.0 to 2.0 and receipt-exported reviews from 16 of 16 to 0 of 16.

Taken together, the ablations indicate that each stage contributes a distinct review function. The evidence chain therefore behaves as a layered architecture rather than as a single monolithic switch.

## 4.3 Falsification resistance

Table 3 reports the strengthened negative-control suite. Seven scenarios are derived from the 11 evidence-chain bundles that originally reviewed as *pass*, and one scenario is derived from the 3 evidence-chain bundles that originally reviewed as *policy_blocked*. Every derived bundle is downgraded to *partial*, yielding a 1.0 detection rate across all eight falsification scenarios.

The pattern of failure is selective rather than indiscriminate. Missing intent clears only the intent verdict. Missing policy clears only the policy verdict. Receipt forgery and receipt swap each clear only the receipt verdict, while payload tampering breaks execution verification and receipt verification together because the recomputed digests no longer match the stored bundle. Cross-run replay mismatch clears execution verification while leaving receipt binding intact, which separates replay-chain integrity from receipt binding. A false `tamper_detected` claim is also rejected in all 11 pass-derived cases.

The policy-bypass claim scenario is the strongest new check. In all 3 source bundles, intent capture, policy persistence, execution verification, and receipt export still remain individually positive after falsification, but the bundle is nonetheless rejected because the claimed completed outcome contradicts the persisted blocked policy decision. This shows that the review contract is no longer limited to field presence and digest recomputation; it now also rejects cross-file contradictions that would otherwise masquerade as valid evidence.

These checks do not solve the external-benchmark problem, but they do tighten one internal-validity concern. The study is no longer only scoring architecture-aligned positives against architecture-aligned metrics; it also demonstrates that the same review contract rejects malformed, transplanted, or over-claimed bundles derived from previously valid runs.

# 5 Related Work

This work sits at the intersection of runtime observability, provenance and audit modeling, policy enforcement, and agent-framework execution tooling. The paper does not attempt to replace those lines of work. Instead, it argues that agentic software systems need an additional bridge layer that turns runtime traces into bounded evidence bundles for independent review.

Runtime observability systems standardize logs, metrics, and traces as the primary surface for inspection. OpenTelemetry is representative of this design center [6], while framework-specific tooling such as LangSmith observability and CrewAI tracing exposes agent traces, callbacks, and task timelines for debugging and operations [2, 3]. These systems make runs inspectable, but they do not by default provide a review contract that explicitly preserves intent capture, policy persistence, integrity status, and receipt export.

Provenance research emphasizes lineage, responsibility, and reproducibility across heterogeneous systems. PROV-O remains a foundational provenance vocabulary for interoperable provenance interchange [4], and recent work such as PROV-AGENT extends this tradition to agentic workflows by capturing prompts, responses, and decisions as workflow provenance [7]. Our execution-evidence layer is narrower than full provenance graphs: it compresses the run into a bounded bundle designed specifically for third-party review questions.

Policy systems show that authorization can be externalized and made explicit. Open Policy Agent is a widely used example of policy-as-code infrastructure [5]. Recent receipt-oriented work also argues that lightweight execution receipts can offer a practical middle ground between unstructured logs and heavyweight cryptographic proof systems for interactive agents [1]. The present paper adopts that general intuition, but evaluates it at the level of exported run bundles, review logic, and controlled architectural ablations.

Relative to these strands, the claim of this paper is modest. Observability surfaces, provenance models, and policy engines each solve part of the problem, but they do not by default provide a bounded five-file review bundle plus a four-question independent review contract evaluated through controlled ablations and falsification checks. The operational novelty claimed here is therefore not telemetry, provenance, or policy evaluation in isolation; it is that

combined review surface. The proposed architecture is intended as that bridge, not as a replacement for tracing, provenance, or policy tooling.

## 6 Discussion and Threats to Validity

The reported results support a bounded systems claim: execution evidence can be modeled as a distinct layer above runtime observability and below downstream audit interpretation. In the implemented harness, this layer improves pre-execution explicitness, preserves governance outcomes as evidence-bearing states, detects tamper scenarios as explicit review results, exports bounded receipts that a third party can inspect without reading a full trace transcript, and rejects falsified bundle variants derived from previously valid runs.

The results do not show semantic correctness of model outputs, formal verification of arbitrary agent behavior, or a complete representation of any production framework. The paper should therefore be read as a reference-architecture and reference-implementation study, not as a claim of universal runtime coverage.

Threats to validity remain. The metrics are evidence-shape metrics rather than semantic task-quality metrics. The harness is deterministic and local, so it does not exercise the nondeterminism, service failures, and integration drift of production agent stacks. The live framework baselines are minimal integrations, not production benchmarks, although the paired CrewAI and LangChain comparisons materially reduce the straw-baseline concern. The falsification suite also remains bounded: it strengthens adversarial validity by checking omission, replay mismatch, receipt swap, policy-bypass claims, payload tampering, and false tamper claims, but it does not cover open-ended attacks or broader live-service adversaries. Persona projection likewise remains the least validated stage and is treated here as an architectural precondition rather than an independently established causal factor.

Despite these limits, the reference path is useful for software engineering practice. It suggests that agentic systems need a distinction between *runtime telemetry*, which helps operators observe a system, and *execution evidence*, which helps reviewers verify what the system claimed to do.

## 7 Conclusion

This paper presented an execution evidence architecture for agentic software systems and instantiated it as a reproducible reference path in a submitted artifact package omitted here for double-blind review. The central claim is that observability alone is not enough: agent runs need structured evidence bundles that make intent, authorization, execution verification, and exported receipts independently reviewable.

The submitted evaluation harness supports a bounded version of that claim with a deterministic 16-task suite, a stable five-file export contract, a third-party review script, minimal live CrewAI and LangChain baselines, same-framework paired comparisons, four ablation modes, and a small negative-control falsification suite. Across the current summaries, the full evidence chain preserves review questions that the trace-oriented baseline and the live framework observability baselines do not explicitly retain, the same live runtime becomes more reviewable when wrapped by the evidence layer in two framework ecosystems, the architecture shows interpretable degradation under targeted ablations, and the same review contract rejects malformed or over-claimed bundles derived from previously valid runs.

Future work should validate the architecture against broader, production-grade live framework integrations, extend the integrity layer beyond the current bounded checks, and run a real blinded reviewer study. Even in its current form, the reference implementation shows a plausible and reproducible path for engineering execution evidence as a first-class systems layer rather than treating it as a side effect of logging.

## Data Availability Statement

An anonymized artifact package has been prepared for review and includes the task suite, exported bundles, metric summaries, and scripts needed to regenerate the reported tables and rebuild the manuscript. To preserve double-anonymous review, public archival metadata and DOI release are withheld at submission time. DOI-based public archival will be completed through the submission workflow or camera-ready process, subject to venue policy.

## References

[1] Abhinaba Basu. 2026. Tool Receipts, Not Zero-Knowledge Proofs: Practical Hallucination Detection for AI Agents. https://arxiv.org/abs/2603.10060. Accessed: 2026-03-13.

[2] CrewAI. 2026. CrewAI Tracing. https://docs.crewai.com/en/observability/tracing. Accessed: 2026-03-13.

[3] LangChain. 2026. LangSmith Observability. https://docs.langchain.com/langsmith/observability. Accessed: 2026-03-13.

[4] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. 2013. PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/. W3C Recommendation.

[5] Open Policy Agent Authors. 2026. Open Policy Agent Documentation. https://www.openpolicyagent.org/docs. Accessed: 2026-03-13.

[6] OpenTelemetry Authors. 2026. What is OpenTelemetry? https://opentelemetry.io/docs/what-is-opentelemetry/. Accessed: 2026-03-13.

[7] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. 2025. PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. In *2025 IEEE International Conference on eScience (eScience)*. doi:10.1109/eScience65000.2025.00093